

Assignments

1. Accept a char input from the user and display it on the console.

```
x = input("Enter a char: ")  
  
print(x)
```

1. Accept two inputs from the user and output its sum.

Variable	Data Type
Number 1	Integer
Number 2	Float
Sum	Float

```
x = int(input("Enter a num1: "))  
  
y = float(input("Enter a num2: "))  
  
print(float(x+y))  
x = int(input("Enter a num1: "))
```

```
y = float(input("Enter a num2: "))  
  
print(float(x+y))
```

1. Write a program to find the simple interest.

- a. Program should accept 3 inputs from the user and calculate simple interest for the given inputs. Formula: $SI = (P * R * n) / 100$

Variable	Data Type
Principal amount (P)	Integer
Interest rate (R)	Float
Number of years (n)	Float
Simple Interest (SI)	Float

```
p = int(input("Enter Principal amount : "))  
  
i = float(input("Enter Interest rate: "))  
  
n = float(input("Enter Number of years: "))
```

```
si = ((p*i*n)/100)

print(f"The simple intrest is {si}")
```

1. Write a program to check whether a student has passed or failed in a subject after he or she enters their mark (pass mark for a subject is 50 out of 100).

- a. Program should accept an input from the user and output a message as “Passed” or “Failed”

Variable	Data type
mark	float

```
mark = int(input("Enter Your Marks: "))

if mark >= 50:
    print("Passed")
elif mark < 50:
    print("Failed")
else:
    print("Invalid Entry!")
```

1. Write a program to show the grade obtained by a student after he/she enters their total mark percentage.
 - a. Program should accept an input from the user and display their grade as follows

Mark	Grade
> 90	A
80-89	B
70-79	C
60-69	D
50-59	E
< 50	Failed

Variable	Data type
Total mark	float

```

mark = float(input("Enter your total mark percentage: "))

if mark >= 90:
    print("Grade: A")
elif mark >= 80:
    print("Grade: B")
elif mark >= 70:
    print("Grade: C")
elif mark >= 60:
    print("Grade: D")
elif mark >= 50:
    print("Grade: E")
else:
    print("Failed")

```

1. Using the 'switch case' write a program to accept an input number from the user and output the day as follows.

Input	Output
1	Sunday
2	Monday
3	Tuesday
4	Wednesday
5	Thursday
6	Friday
7	Saturday
Any other input	Invalid Entry

```
day = int(input("Enter a number (1-7): "))
```

```
match day:
```

```
    case 1:
```

```
        print("Sunday")
```

```
    case 2:
```

```
        print("Monday")
```

```
    case 3:
```

```
        print("Tuesday")
```

```
    case 4:
```

```
        print("Wednesday")
```

```
    case 5:
```

```
        print("Thursday")
```

```
    case 6:
```

```
        print("Friday")
```

```
    case 7:
```

```
        print("Saturday")
```

```
    case _:
```

```
        print("Invalid!")
```

1. Write a program to print the multiplication table of given numbers. Using for and while
 - a. Accept an input from the user and display its multiplication table

Eg:

Output: Enter a number

Input: 5

Output:

1 x 5 = 5

2 x 5 = 10

$$3 \times 5 = 15$$

$$4 \times 5 = 20$$

$$5 \times 5 = 25$$

$$6 \times 5 = 30$$

$$7 \times 5 = 35$$

$$8 \times 5 = 40$$

$$9 \times 5 = 45$$

$$10 \times 5 = 50$$

```
num = int(input("Enter the number: "))  
  
for i in range(1, 11):  
    multi = num * i  
    print(f"{num} X {i} = {multi} \n")
```

1. Write a program to print the following pattern (**hint**: use nested loop)

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

```
n = 5
```

```
for i in range(1, n+1):  
    for j in range(1, i+1):  
        print(j, end=" ")  
    print("\n")
```

1. Write a program to create a copy of array and add an element to copied array . show both arrays.

- a. Program should accept an array from the user, swap the values of two arrays and display it on the console

Eg: **Output:** Enter the size of Array 1

Input: 5

Output: Enter the values of Array 1

Array 1: 10, 20, 30, 40, 50

Array 2 : Copy of Array 1

Array 2 : 10, 20, 30, 40, 50 + add a element

Output : Array 1 [10, 20, 30, 40, 50]

Output : Array 2 [10, 20, 30, 40, 50, 60, 70]

```
n = int(input("Enter the size of the array: "))  
  
array1 = []  
array2 = []
```



```
for i in range(n):
    value = int(input("Enter the elements: "))
    array1.append(value)

array2 = array1.copy()

array2.append(20)
array2.append(30)

print(array1, array2)
```

1. Write a program to sort an array in descending order without sort() and sorted()
Program should accept an array, sort the array values in descending order and display it

- Selection
- Insertion
- bubble

Eg: **Output:** Enter the size of an array

Input: 5

Output: Enter the values of array

Input: 20, 10, 50, 30, 40

Output: Sorted array:

50, 40, 30, 20, 10

```
n = int(input("Enter a size: "))

array = []
```

```
for i in range(n):
    value = int(input("Enter the elements: "))
    array.append(value)

for i in range(n):
    for j in range(n - i - 1):
        if array[j] < array[j+1]:
            array[j], array[j+1] = array[j+1], array[j]

print("The Sorted Array: ", array)
```

1. Write a program to identify whether a string is a palindrome or not without using reverse(), slicing

- a. A string is a palindrome if it reads the same backward or forward eg:
MALAYALAM

Program should accept a string and display whether the string is a
palindrome or not

Eg: **Output:** Enter a string

Input: MALAYALAM

Output: Entered string is a palindrome

Eg 2: **Output:** Enter a string

Input: HELLO

Output: Entered string is not a palindrome

```
text = input("Enter a string: ")

temp = text

reverse = ""
```

```
for ch in text:
    reverse = ch + reverse

if text==reverse:
    print("Paliondrome")
else:
    print("Not Paliondrome")
```

1. Write a program to add to two dimensional arrays, understand the memory management of list

- a. Program should accept two 2D arrays and display its sum

Eg: **Output:** Enter the size of arrays

Input: 3

Output: Enter the values of array 1

Input:

1 2 3

4 5 6

7 8 9

Output: Enter the values of array 2

Input:

10 20 30

40 50 60

70 80 90

Output: Sum of 2 arrays is:

11 22 33

44 55 66

77 88 99

```
size = int(input("Enter the size of arrays: "))

array1 = []
array2 = []
sum_array = []

print("Enter the values of Array 1:")
for i in range(size):
    row = []
    for j in range(size):
        row.append(int(input()))
    array1.append(row)

print("Enter the values of Array 2:")
for i in range(size):
    row = []
    for j in range(size):
        row.append(int(input()))
    array2.append(row)

# Add the two arrays
for i in range(size):
    row = []
    for j in range(size):
        row.append(array1[i][j] + array2[i][j])
    sum_array.append(row)
```

```
print("Sum of 2 arrays is:")  
for row in sum_array:  
    print(*row)
```

1. Grades are computed using a weighted average. Suppose that the written test counts 70%, lab exams 20% and assignments 10%.

If Arun has a score of

Written test = 81

Lab exams = 68

Assignments = 92

Arun's overall grade = $(81 \times 70)/100 + (68 \times 20)/100 + (92 \times 10)/100 = 79.5$

Write a program to find the grade of a student during his academic year.

- a. Program should accept the scores for written test, lab exams and assignments
- b. Output the grade of a student (using weighted average)

Eg:

Enter the marks scored by the students

Written test = 55

Lab exams = 73

Assignments = 87

Grade of the student is 61.8

```
written = int(input("Enter the marks or Written Exam: "))  
  
lab = int(input("Enter the marks or Lab Exam: "))  
  
assignments = int(input("Enter the marks or Assignments: "))  
  
grade = (written*70)/100 + (lab*20)/100 + (assignments*10)/100  
  
print(grade)
```

1. *Study about functions*

□ *User defined*

- *Types of Arguments*

□ *Lambda*

Write a program using user defined functions and lambda functions

```
def add(a, b):  
    return a + b  
  
func = lambda x: x**2  
  
print(add(2, 5))  
print(func(2))
```

1. Write a program to print “HELLO WORLD “using function without using print inside of function. (“HELLO WORLD “must be inside Decorator function)

```
def decorator(func):  
    def wrapper():  
        return "Hello"  
    return wrapper
```

```
@decorator
def msg():
    pass

print(msg())
```

1. Write a menu driven program to do the basic mathematical operations such as addition, subtraction, multiplication and division (**hint**: use if else ladder or switch)
 - a. Program should have 4 functions named addition(), subtraction(), multiplication() and division()
 - b. Should create a class object and call the appropriate function as user prefers in the main function

```
x = int(input("Enter num1: "))
y = int(input("Enter num2: "))

op = input("Enter the operation(+, -, *, /): ")

match op:
    case "+":
        print(x + y)
    case "-":
        print(x - y)
    case "*":
        print(x * y)
    case "/":
        print(x/y)
    case _:
        print("Invalid")
```

1. Write a program to print the following pattern using for loop

7 8 9 10

4 5 6

2 3

1

```
num = 7

for i in range(4, 0, -1):
    for j in range(i):
        print(num, end=" ")
        num += 1
    print()
    num = num - i - (i - 1)
```

1. Write a program to add the values of two 2D arrays
 - a. Program should contain a class, Functions should be inside the class
 1. Call function `getArray()` using an object
 2. Call function `addArray()` using an object
 3. Call function `displayArray()` using an object

getArray()

1. Get values to the array

getArray()

1. Add array 1 and array 2

displayArray()

1. Display the array values

Eg:

Enter the size of array

2

Enter the values of array 1

1 2

3 4

Enter the values of array 2

5 6

7 8

Output:

Sum of array 1 and array 2:

6 8

10 12

```
class ArrayOperation:

    def getArray(self):
        self.size = int(input("Enter the size of array: "))

        self.array1 = []
        self.array2 = []

        print("Enter the values of array 1:")
        for i in range(self.size):
            row = list(map(int, input().split()))
            self.array1.append(row)
```

```

        print("Enter the values of array 2:")
        for i in range(self.size):
            row = list(map(int, input().split()))
            self.array2.append(row)

    def addArray(self):
        self.sumArray = []

        for i in range(self.size):
            row = []
            for j in range(self.size):
                row.append(self.array1[i][j] + self.array2[i][j])
            self.sumArray.append(row)

    def displayArray(self):
        print("Sum of array 1 and array 2:")
        for row in self.sumArray:
            print(row)

obj = ArrayOperation()

obj.getArray()
obj.addArray()
obj.displayArray()

```

1. Write a program to include all the functionalities of a calculator using ABSTRACT class and abstract method. All the methods (add, sub, mul, div) should be inside of abstract class. Abstract method definition should be in another class.

Examples :

```

from abc import ABC, abstractmethod

class Calculator(ABC):
    def __init__(self, id, name):
        self.id = id
        self.name = name

    @abstractmethod

```

```
def add (self):  
    pass
```

```
from abc import ABC, abstractmethod  
  
class Caluculations(ABC):  
  
    @abstractmethod  
    def add(self, x, y):  
        pass  
  
    @abstractmethod  
    def sub(self, x, y):  
        pass  
  
    @abstractmethod  
    def multi(self, x, y):  
        pass  
  
    def div(self, x, y):  
        pass  
  
class Calculators(Caluculations):  
  
    def add(self, a, b):  
        print("Sum =", a + b)  
  
    def sub(self, a, b):  
        print("Difference =", a - b)  
  
    def mul(self, a, b):  
        print("Product =", a * b)  
  
    def div(self, a, b):  
        if b == 0:  
            print("Division by zero is not possible.")  
        else:
```

```

        print("Quotient =", a / b)

obj = Calculators()

print("1. Addition")
print("2. Subtraction")
print("3. Multiplication")
print("4. Division")

choice = int(input("Enter your choice: "))

a = float(input("Enter first number: "))
b = float(input("Enter second number: "))

match choice:
    case 1:
        obj.add(a, b)
    case 2:
        obj.sub(a, b)
    case 3:
        obj.mul(a, b)
    case 4:
        obj.div(a, b)
    case _:
        print("Invalid Choice")

```

1. Write a program to build a home. The Home class should define all the attributes of each room in a home. From the Home class create two homes. FirstHome and SecondHome. First home should have an extra study room as a method. SecondHome should have the work_area as an extra method. should use the concept of inheritance.

```

class Home:
    def __init__(self):
        pass
    def room1:
        width=100
        breadth = 100

```

```
    print('are of room1',width*breath)

def kitchen:
    width = 1222
    breadth = 4888
    print('are of kitchen',width*breath)
```

you should have mentioned all the plans of home here as methods.

```
Class FirstHome(Home):
    # define the extra method's
    pass

class SecondHome(Home):
    # define the extra method's
    pass
```

```
class Home:

    def __init__(self):
        pass

    def bed_room():
        length = 100
        width = 75
        print("Total Area = ", length * width)

    def bathroom():
        length = 50
        width = 50
        print("Total Area = ", length * width)

class Firsthome(Home):
    def __init__(self):
        pass

    def study_room():
```

```

        length = 300
        width = 200
        print("Total Area = ", length * width)

class Secondhome(Home):
    def __init__(self):
        pass

    def work_area():
        length = 300
        width = 200
        print("Total Area = ", length * width)

```

1. *Write a program to create Class with name and account number and implement get and set, with property decorator and making account number and name private.*

```

class BankAccount:

    def __init__(self, name, acc_no):
        self.__name = name
        self.__acc_no = acc_no

    @property
    def name(self):
        return self.__name

    @name.setter
    def name(self, name):
        self.name = name

    @property
    def acc_no(self):
        return self.__acc_no

    @acc_no.setter
    def acc_no(self, acc_no):
        self.__acc_no = acc_no

obj = BankAccount("Rahul", 123456789)

```

```

print("Name:", obj.name)
print("Account Number:", obj.acc_no)

obj.name = "Rashik"
obj.acc_no = 987654321

print("\nAfter Updating:")
print("Name:", obj.name)
print("Account Number:", obj.acc_no)

```

1. Write a function to calculate the sum of all numbers passed as its arguments. Your function should be called `sum_numbers` and should define a single variable argument (i.e. a star argument) that will get the values to sum.

Test the function with the following values:

<i>Values</i>	<i>Result</i>
1, 2, 3	6
8, 20, 2	30
12.5, 3.147, 98.1	113.747
1.1, 2.2, 5.5	8.8

```

def sum_numbers(*numbers):
    total = 0

    for num in numbers:
        total += num

    return total

result = sum_numbers(10, 20, 30, 40, 50)

print("Sum =", result)

```

```
1. pantry = {  
    "chicken": 500,  
    "lemon": 2,  
    "cumin": 24,  
    "paprika": 18,  
    "chilli powder": 7,  
    "yogurt": 300,  
    "oil": 450,  
    "onion": 5,  
    "garlic": 9,  
    "ginger": 2,  
    "tomato puree": 125,  
    "almonds": 75,  
    "rice": 500,  
    "coriander": 20,  
    "lime": 3,  
    "pepper": 8,  
    "egg": 6,  
    "pizza": 2,  
    "spam": 1,  
}
```

```
recipes = {  
    "Butter chicken": [  
        "chicken",  
        "lemon",  
        "cumin",  
        "paprika",  
        "chilli powder",  
        "yogurt",
```


*"oil",
"onion",
"garlic",
"ginger",
"tomato puree",
"almonds",
"rice",
"coriander",
"lime",*

],

"Chicken and chips": [

*"chicken",
"potatoes",
"salt",
"malt vinegar",*

],

"Pizza": [

"pizza",

],

"Egg sandwich": [

*"egg",
"bread",
"butter",*

],

"Beans on toast": [

*"beans",
"bread",*

],

"Spam a la tin": [

"spam",

```
        "tin opener",
        "spoon",
    ],
}
```

observe the dictionary above and write a menu driven python program to create recipes. Once one recipe is done then the quantity of the items in pantry should also be reduced

Eg : If you cook beans on toast the beans quantity and bread quantity need to decrease i.e., one from the total quantity each.

```
pantry = {
    "chicken": 500,
    "lemon": 2,
    "cumin": 24,
    "paprika": 18,
    "chilli powder": 7,
    "yogurt": 300,
    "oil": 450,
    "onion": 5,
    "garlic": 9,
    "ginger": 2,
    "tomato puree": 125,
    "almonds": 75,
    "rice": 500,
    "coriander": 20,
    "lime": 3,
    "pepper": 8,
    "egg": 6,
    "pizza": 2,
    "spam": 1,
}
```

```
recipes = {
    "Butter chicken": [
        "chicken", "lemon", "cumin", "paprika", "chilli powder",
```

```

        "yogurt", "oil", "onion", "garlic", "ginger",
        "tomato puree", "almonds", "rice", "coriander", "lime"
    ],
    "Chicken and chips": [
        "chicken", "potatoes", "salt", "malt vinegar"
    ],
    "Pizza": [
        "pizza"
    ],
    "Egg sandwich": [
        "egg", "bread", "butter"
    ],
    "Beans on toast": [
        "beans", "bread"
    ],
    "Spam a la tin": [
        "spam", "tin opener", "spoon"
    ],
}

```

```

while True:
    print("\n----- MENU -----")
    recipe_list = list(recipes.keys())

    for i in range(len(recipe_list)):
        print(f"{i + 1}. {recipe_list[i]}")
    print("0. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 0:
        print("Thank You!")
        break

    if 1 <= choice <= len(recipe_list):
        recipe = recipe_list[choice - 1]
        ingredients = recipes[recipe]

        available = True

```

```

        for item in ingredients:
            if item not in pantry or pantry[item] <= 0:
                print(f"Cannot prepare {recipe}. '{item}' is not
available.")
                available = False
                break

        if available:
            for item in ingredients:
                pantry[item] -= 1

            print(f"\n{recipe} prepared successfully!")

            print("\nUpdated Pantry:")
            for item, qty in pantry.items():
                print(f"{item}: {qty}")

        else:
            print("Invalid Choice")

```

1. *create a custom exception class and raise this exception when user press one in the menu and handles this exception.*

```

class AgeError(Exception):
    pass

age = -5

try:
    if age < 0:
        raise AgeError("Invaild Entry")
except AgeError as e:
    print(e)

```

1. *Write a list comprehension that returns the list ["1*2=1", "2*2=4", "3*2=9", ..., "25*2=625"]*

```

list = [f"{i}*{i}={i*i}" for i in range(1, 26)]
print(list)

```

1. *Using dict comprehension and a conditional argument create a dictionary from the current dictionary where only the key:value pairs with value above 2000 will be taken to the new dictionary.*

dict1={"NFLX":4950,"TREX":2400,"FIZZ":1800, "XPO":1700}

dict2={}

```
dict1 = {  
    "NFLX": 4950,  
    "TREX": 2400,  
    "FIZZ": 1800,  
    "XPO": 1700  
}  
  
dict2 = {key: value for key, value in dict1.items() if value > 2000}  
  
print(dict2)
```